# ANÁLISE DO PROJETO

S3 — Alertas e Notificações | Squad VER-3

Revisão do código, testes, arquitetura, lacunas e próximos passos

Base: ZIP enviado pelo usuário; testes executados novamente nesta revisão.

# 1. Resumo executivo

O S3 já tem uma base funcional para processar eventos em memória: valida eventos, resolve regras de roteamento, cria alertas, evita duplicatas na instância atual do repositório, registra histórico e oferece reconhecimento (ACK) e silenciamento (MUTE).

Ainda não é uma integração operacional completa. Não há cliente de barramento real, publicador de saída, persistência durável, despacho efetivo de notificações nem integração ponta a ponta demonstrada.

Testes desta revisão: PASSOU — Ran 56 tests in 0.004s.

# 2. Arquitetura observada

```
AssinanteEventos (interface abstrata; sem broker concreto)\n    ↓ callback\nConectorEventos → IngestorEventos (JSON →
```

Na arquitetura de referência, o S2 produz os eventos qualificados consumidos pelo S3. O código modela uma fronteira de assinatura, mas usa um assinante falso nos testes. Não há publicação real de alertas para A5, A3 ou A7.

# 3. Arquivos e responsabilidades

| Arquivo | Responsabilidade / observação |
|---|---|
| contracts.py | EventoQualificado e validação básica; confiança entre 0 e 1. |
| alert.py | Modelo Alerta: IDs, severidade, destinatários, canais, estado e horário UTC. |
| history.py | RegistroHistorico congelado; ações CRIADO/RECONHECIDO/SILENCIADO/ENCERRADO. |
| repository.py | Alertas e histórico em memória; busca por ID/evento; deduplicação na instância. |
| routing.py | Configuração por tipo: severidade, destinatários, canais SOM/TELA/PUSH. |
| service.py | Criação, roteamento, reconhecimento e silenciamento; sem encerramento. |
| ingestion.py | Decodifica objeto JSON na raiz e encaminha evento. |
| subscription.py | Protocolo abstrato de assinatura e callback; sem broker concreto. |
| demo_s3.py | Demonstração manual com assinante simulado. |
| jsons/*.json | Exemplos de envelope; entrada não compatível com ingestor atual. |

# 4. O que funciona

## Ingestão e validação

• Rejeita JSON inválido e conteúdo que não seja objeto JSON.

• Valida campos obrigatórios e confiança entre 0 e 1.

• Timestamp é verificado apenas como texto não vazio; ISO-8601/fuso horário não são validados.

## Roteamento

• O tipo do evento seleciona uma configuração predefinida de severidade, destinatários e canais.

• Evento sem regra é rejeitado; canais aceitos: SOM, TELA e PUSH.

• A severidade é configurada na regra; não é calculada dinamicamente a partir de confiança, duração ou política de negócio.

## Alertas e histórico

• Estados exercitados: CRIADO, RECONHECIDO e SILENCIADO.

• Histórico registra criação e autor/horário nas ações de reconhecimento/silenciamento.

• ENCERRADO é aceito pelo modelo de histórico, mas não há método para encerrar alertas.

• O histórico é imutável como objeto, mas fica em armazenamento volátil.

• A deduplicação não sobrevive a reinício e não resolve concorrência entre processos.

# 5. Lacunas críticas

| Prio. | Pendência | Impacto |
|---|---|---|
| P1 | Integração B3/Pub/Sub real e contrato oficial | Sem consumo real; transporte, tópico, ack/retry e ordenação indefinidos. |
| P1 | Compatibilidade do envelope | entrada.json tem payload aninhado; ingestor espera campos na raiz. |
| P1 | Publicação de saída | Não há publisher; saida.json é exemplo documental. |
| P1 | Entrada de comandos/autorização | ACK/MUTE só são métodos Python, sem interface de comando. |
| P2 | Persistência durável | Alertas/histórico se perdem ao reiniciar. |
| P2 | Encerramento e transições | Não existe operação de encerramento nem máquina de estados formal. |
| P2 | Idempotência robusta | Deduplicação é só em memória. |
| P2 | Retry, DLQ, backpressure, logs/métricas | Sem tratamento operacional de falhas/sobrecarga. |
| P2 | Despacho SOM/TELA/PUSH | Canais são metadados, não entregas efetivas. |
| P3 | Validações adicionais | Timestamp, severidade, estado e campos do contrato precisam ser formalizados. |

# 6. Observações técnicas

• Os testes com assinante falso validam o callback interno, não a integração real com o barramento.

• entrada.json usa message_type/schema/schema_version/producer/published_at/payload. O ingestor tenta construir EventoQualificado a partir da raiz inteira, então esse exemplo falha sem adaptação.

• saida.json não é produzido pelo código; não existe publicador de alertas.

• reconhecer_alerta não bloqueia explicitamente um novo reconhecimento quando o alerta já está RECONHECIDO.

• O repositório devolve objetos Alerta mutáveis; concorrência exige uma política explícita.

• O ZIP contém __pycache__ e .pyc que não deveriam normalmente ser versionados.

# 7. Como executar

```
PowerShell na raiz do projeto:\n\n$env:PYTHONPATH = "src"\npython -m unittest discover -s tests/s3 -p "test_*.py" -v\np
```

Resultado desta revisão: PASSOU — Ran 56 tests in 0.004s.

# 8. Plano recomendado

## Etapa 1 — fechar contratos

• Acordar com B3/TRA-1 transporte, tópico, envelope, versão, confirmação, retry, ordenação e erros.

• Adaptar o ingestor ao envelope oficial ou corrigir os exemplos JSON.

• Definir contrato de saída e destinatários.

## Etapa 2 — integrar

• Implementar adaptador real de barramento atrás de AssinanteEventos.

• Implementar publicação de alertas e entrada de comandos ACK/MUTE com autorização.

• Criar testes de integração com o transporte escolhido.

## Etapa 3 — confiabilidade

• Persistir alertas e histórico em banco.

• Implementar encerramento e transições formais.

• Garantir idempotência durável, retry, dead-letter, backpressure, logs e métricas.

• Integrar os canais de notificação ou delegar formalmente o despacho.

### Etapa 4 — aceite

• Evento oficial do S2 gera exatamente um alerta.

• Reentrega não duplica alertas/histórico.

• ACK/MUTE autorizado registra autor e horário.

• Dados sobrevivem a reinício.

• Alerta publicado no contrato acordado é consumido por componente real.

• Teste ponta a ponta cobre falhas, retry e recuperação.

## 9. Limites da revisão

Esta análise se baseia no ZIP enviado e nos testes executados nesta revisão. Não confirma contratos externos, infraestrutura ou estado dos demais componentes. Capacidades não demonstradas foram registradas como pendentes.